Ground Control — Demo Run of Show

2 intro slides + the FULL ./demo.sh (8 beats) · target ~2:58 · hard cap 3:00 · each beat says WHAT it does and WHY it matters

Pacing is tight. Move briskly and press Enter promptly. **Trim levers if long:** drop the slides, or skip Beat 7 (JSON) and show only Beat 7b (the bundle).

PART 1 — SLIDES (0:00–0:24) present intro-slides.pptx, keep brief

Slide 1 — The Problem (0:00–0:12)

SAY "A thousand-GPU training run slows down — or dies. Finding out why is hours of hand-digging through logs and counters across many nodes while the GPUs sit idle — and it looks the same whether it's bad hardware or one wrong setting."

Slide 2 — The Solution (0:12–0:24)

SAY "Ground Control: one command names the root cause and the fix in 30 seconds — for ops, customers, and support. Let me show you." → switch to the terminal.

PART 2 — LIVE DEMO (0:24–~2:58) run: ./demo.sh

Intro banner (0:24–0:28)

WHAT starts the demo runner; prints the one-line problem banner.

WHY hand-off from the slides to the real tool.

SAY "Here it is, live."

DO `press Enter.`

Beat 1 — The catalog (0:28–0:40)

audit -l

WHAT lists all 51 checks (groups A/B/C/G + NCCL) and the use-case bundles.

WHY breadth, and it auto-detects the hardware — checks that don't apply show N-A instead of firing false alarms.

SAY "51 checks across the server, the network, and the GPUs. It auto-detects the hardware, and bundles the right checks per symptom."

`DO press Enter.`

Beat 2 — A healthy node (0:40–0:54)

audit -u preflight --fixture fixtures/cw0bec_blackwell_healthy

WHAT runs the preflight bundle against a REAL capture from a live CoreWeave Blackwell node.

WHY proves it runs on real hardware; all-green + exit 0 can gate CI.

SAY "Real data from a live CoreWeave Blackwell node — eight GPUs, nine network cards. All green, exit zero: you can gate CI and stop a bad node before it ever gets a job."

`DO press Enter.`

Beat 3 — A failing GPU (0:54–1:10)

audit -u gpu_fault --fixture fixtures/synth_gpu_fault

WHAT runs the GPU-fault bundle on a failing-GPU capture.

WHY shows correlation — three separate signals on ONE GPU become a single verdict with an action.

SAY "First, a failing GPU. Three separate signs on the same GPU — driver errors, failing

memory, and ninety-one degrees — collapse into one verdict: fence the node off and replace the card.”

> **DO** press Enter.

> Beat 4 — NCCL on TCP · THE HEADLINE (1:10–1:42 · slow down)

> audit -u nccl_slow --fixture fixtures/nccl_tcp_fallback

> **WHAT** runs the NCCL-slow bundle against real NCCL logs where NCCL fell back to
> TCP.

> **WHY** the signature SILENT failure — a ~100× slowdown that just looks like “the cluster
> is slow”; the verdict names the cause + the fix. The differentiator.

SAY “Now the big one. GPUs sync data during training through a library called NCCL. Normally it uses RDMA — the fast path where network cards talk directly and skip the CPU. One wrong setting silently dropped it to ordinary TCP — about a hundred times slower — looking just like ‘the cluster is slow.’ The verdict names the cause AND the exact fix. That’s the moonshot: an answer, not another dashboard.”

> **DO** press Enter.

> Beat 5 — A dead RDMA port (1:42–1:58)

> audit -u preflight --fixture fixtures/synth_fabric_fault

> **WHAT** runs preflight on a fabric-fault capture (a down port, a port throwing errors, a
> congested port).

> **WHY** shows it correlates NETWORK faults too, and names the specific bad link.

SAY “A dead network port. One port down, another throwing errors, a third congested — the verdict pins the dead rail and the action: check the cable, drain the node.”

DO press Enter.

Beat 6 — Customer profile (1:58–2:14)

audit --profile customer -u nccl_slow --fixture fixtures/nccl_tcp_fallback

WHAT the same NCCL run, through the customer profile.

WHY one engine, three audiences; only safety, redaction, and plain-English remediation change.

SAY “Same engine, three audiences. In customer mode it’s read-only, internal names are hidden, and every failure carries a plain-English fix and a ‘contact support’ line.”

DO press Enter.

Beat 7 — Support · JSON (2:14–2:26 · TRIM LEVER: skippable)

audit --profile support -u gpu_fault --fixture fixtures/synth_gpu_fault --json | head -20

WHAT emits structured JSON — verdict, severity, scope, supporting checks.

WHY machine-readable for pipelines / CI / dashboards.

SAY “For support and automation, the same run emits structured JSON — the verdict, how severe, where, and the checks behind it.”

DO press Enter.

Beat 7b — Support · the bundle (2:26–2:44)

audit --profile support -u gpu_fault --fixture fixtures/synth_gpu_fault --bundle

WHAT writes a redacted tarball (result.json + raw evidence).

> **WHY** one artifact a customer or engineer attaches straight to a ticket.

SAY “And a redacted bundle — the result plus the raw evidence — to attach straight to a support ticket. Self-serve, into a clean handoff.”

> **DO** `press Enter.`

> Close (2:44–2:58)

> **WHAT** the summary banner.

> **WHY** land the value and the moonshot.

SAY “Faster runs and less idle spend for customers; fewer escalations and higher fleet use for CoreWeave. And the foundation to predict these failures before they strike. Ground Control.”

Keep under 3:00. Rubric — Speed: Slide 1 & Beat 4 · Business: Slides, Beats 6–7b, Close · Technical: Beats 1–5 · Presentation: the whole Problem → Solution → Prototype → Impact arc.